Distributed Federated Learning

Project Write-up

Kspsvln Siddardha Kumar Kavuri (20250201061)
Sankalp Chaturvedi (20250201083)

Project Description

This project implements a federated learning system consisting of a central coordinator and 4–6 worker nodes, each holding a private, non-shared local data shard. Using a small supplied classifier and dataset, the system trains a global model across multiple communication rounds: in each round, the coordinator broadcasts the current global model to all participating workers, each worker trains locally on its own data shard, and the coordinator aggregates the resulting model updates using sample-weighted Federated Averaging (FedAvg). The project explores both uniform (IID) and non-uniform (non-IID) data partitioning schemes, including settings where individual workers observe only a subset of the available classes, to study how data heterogeneity affects convergence. Round-level robustness is incorporated through round identifiers and timeouts, so that late-arriving updates from a given round are rejected and a round can still complete with a specified minimum level of participation, simulating partial failures and stragglers common in real distributed systems.

Goals

- Implement a working federated learning pipeline with a coordinator and 4–6 worker nodes performing round-based training and sample-weighted FedAvg aggregation.
- Design and evaluate both IID and non-IID data splits across workers, including class-skewed shards for select workers.
- Introduce round identifiers and timeout-based staleness handling, allowing rounds to proceed under a configurable minimum participation threshold.
- Compare the accuracy and convergence behavior of the federated model against a centrally trained baseline on the same dataset.
- Measure and report system-level metrics: communication cost (bytes transferred per round), convergence speed (rounds/time to target accuracy), and robustness under slow or crashed workers.
- Analyze the trade-offs between communication efficiency, data heterogeneity, and model accuracy inherent to federated learning.

Tools and Technologies

- **Programming Language:** Python
- **Machine Learning Framework:** PyTorch (for the classifier model, local training loops, and parameter aggregation)
- **Communication / Coordination:** gRPC or Python's `socket`/`multiprocessing` module for coordinator–worker message passing (model broadcast, update upload, round control signals); alternatively, a lightweight message queue (e.g., ZeroMQ) may be used
- **Dataset:** A standard small-scale benchmark dataset such as MNIST or CIFAR-10, partitioned into IID and non-IID shards across workers

- **Concurrency/Simulation:** Multi-process or multi-threaded simulation of workers on a single machine, with artificial delay/failure injection to emulate stragglers and crashes
- **Monitoring and Analysis:** Logging utilities and Python plotting libraries (Matplotlib/Pandas) for tracking accuracy, convergence curves, and bytes-per-round statistics
- **Version Control:** Git/GitHub for collaborative development

Special System Requirements

No specialized computing resources (e.g., RCE/cloud clusters) are required for this project. The entire system, including the coordinator and all 4–6 simulated workers, will run locally as separate processes (or lightweight containers) on standard development machines, communicating over local sockets/RPC. The datasets and classifier models chosen are intentionally small-scale so that training can be completed on CPU within a reasonable time; GPU acceleration, if available, may be used optionally to speed up experimentation but is not a requirement.

Expected Outcomes

- A functioning federated learning simulation with a coordinator and 4–6 workers capable of running multiple rounds of FedAvg-based training to convergence.
- Comparative accuracy results between the federated model (under both IID and non-IID splits) and a centrally trained baseline, quantifying the accuracy gap caused by data heterogeneity.
- Empirical measurements of communication cost (bytes per round) and convergence rate (rounds/time to reach target accuracy) across different configurations.
- A demonstrated mechanism for handling stragglers and crashed workers via round timeouts and minimum-participation thresholds, with an analysis of how partial participation affects final model quality.
- A final report/presentation summarizing the system design, experimental results, and insights into the cost-accuracy-robustness trade-offs of federated learning.